Requirement on academic honesty (COMP2090SEF):

1. Each group must make the following declaration on the cover page of the report

Hong Kong Metropolitan University

COMP2090SEF
Data Structures, Algorithms And Problem Solving

CourseStudyReport

Group 105

Submission date: 17/4

We declare that: (i) all members of the group have read and checked that all parts of the project (including proposal, code programs, and reports), irrespective of whether they are contributed by individual member or all members as a group, here submitted is original except for source material explicitly acknowledged; (ii) the project, in parts or in whole, has not been submitted for more than one purpose without declaration; (iii) we are aware of the University's policy and regulations on honesty in academic work and understand the possible consequence when breaching such policy and regulations; (iv) we confirm that we have declared in the report about the usage of AI and other generative models, including but not limited to ChatGPT, LLaMA, Gemini, Mistral, and Stable Diffusion, and complied with the instructions provided by HKMU; and (v) we are aware that all members of the group should be held responsible and liable to disciplinary actions, irrespective of whether he/she has signed the declaration and whether he/she has contributed, directly or indirectly, to the problematic contents.

We confirm that we have read through and understood the project requirements. We understand that failure to comply with the project requirements will result in score deduction.

NAME	SID
LI Zhengxuan 李政轩	14037890
CHENG Siu Fung 鄭少峰	14074189
CHAN Chun Sing 陳駿陞	14259934

2. The group must acknowledge ALL external resources or code from others referenced. We will only mark the parts that are original, not the parts from others.

3. All members are liable to the plagiarism incidence, even if a free-rider.

Introduction

In the lecture notes, we have learnt the basic of data structures, such as queue and stack. However, in the reality, we need more precise way to complete our purposes and missions. For example, navigation, mission priorities. We want to process with prioriation but not only the order. Therefore, we chose a data structure that is great for accomplish our goal. In this report, we will study Priority Queue as a Abstract Data Type. And show how it works in Dijkstra's Algorithm. It is a useful tool for solving single-source shortest path problems.

Data Structure: Priority Queue ADT

The basic logic of Priority Queue is different from standard data structures. While queue follows the first-in-first-out principle, the Priority Queue sorting the variables by the priority values. For example, in a min-priority queue, the variable with the smallest priority value will be sorted first. It can be very useful when there are important variables added rather than forcing you to execute the first variable or the last variable like queue and stack.

Key Operations

The Priority Queue mainly supports the following operations:

insert(item, priority): Adds an varibale to the collection with a priority level.

extract_min(): Removes and returns the variable with the highest priority.

peek(): Returns the highest priority varialbe without removing it from the structure.

Implementation

When we use Priority Queue, we use Heap instead of Linked List. The reason we do not use Linked List is that every time we insert a variable to the list, it will scan the list itself causing an increased amount of time which is unnecessary of operational time. To encounter this, we decided to use heap instead of linked list.

The heap we used here is actually a binary heap, it is a complete binary tree satisfying the heap property. Each node is smaller than or equal to its children.
It allows us to find the minimum variable instantly on the root(Sedgewick & Wayne, 2011). This can reduce the time caused by just performing bubble-up or bubble-down process when we add the variables.

Algorithm: Dijkstra's Algorithm

Dijkstra's Algorithm is an algorithm we found useful in the priority queue, the process are shown in below.

1. Set the distance of the starting node to 0 and all other nodes to infinity.
2. Insert the starting node into a priority queue.
3. Extract the node with the smallest distance from the Priority Queue.
4. Calculate the distance through the current node. update the distance if it is shorter than the recorded distance.
5. Add the updated distance to the priority queue and repeat until the it is empty.

The priority queue here is an important role in Dijkstra's algorithm, without it, the algorithm need to search every nodes of the list to find the minimum, if the list is too large, it is an unignorable disadvantage in operation time.

Complexity

Dijkstra's algorithm efficiency is very depend on the data structure we use in the priority queue. When we use binary heap, the total time complexity is O((V + E) log V)(Cormen et al., 2022).

V(Vertices): Represents the number of nodes.
E(Edges): Represents the number of connections between nodes.

The binary heap must execute an operation for re-ording to maintain the priority property, that is where the log V exists to represent when every node is extracted. If it is using a simple list, the total time complexity will be $O(V^2)$ which is much slower than the binary heap.

Conclusion

This study mainly discover the new data structure and algorithm, we have learnt the importance of how choosing the correct data structures and algorithm (Priority Queue, Binary Heap, Dijkstra's Algorithm) can impact the time complexity heavily. This research has taught us the understanding of matching the algorithm and the mathematical models, as well as writing and finding new codes.

GitHub Link:

https://github.com/owounnamed/COMP-2090SEF-GRP_105-project

Appendix

Reference:

Brilliant.org. (n.d.). *Dijkstra's algorithm.*

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik, 1*(1), 269–271.

GeeksforGeeks. (2024). *Priority queue | Set 1 (Introduction)*.

Liskov, B., & Zilles, S. (1974). Programming with abstract data types. *ACM SIGPLAN Notices, 9*(4), 50–59.

Python Software Foundation. (n.d.). heapq — *Heap queue algorithm.* Python 3.12.3 Documentation.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.